

closure_evidence_20260925

BOB-223 closure evidence — 2026-09-25

Revision: 1 Last modified: 2026-09-25T14:10:00Z

Item: BOB-223 — “§11.4.18 script-documentation and §11.4.44 revision headers are ungated — and the perverse consequence is that WRITING the mandated companion doc is what breaks the build”. Duplicate pair: BOB-151 (thinner investigation of the same defect class — the conductor closes it as duplicate-of BOB-223 once this fix lands; this dispatch does not touch the tracker itself).

1. Control-needle verification (item’s own claims, independently re-checked)

```
$ grep -c "CM-SCRIPT-DOCS-SYNC" scripts/  
pre_build_verification.sh    (pre-fix)  
0  
$ grep -c "CM-MARKDOWN-EXPORT-SYNC" scripts/  
pre_build_verification.sh  
6
```

Confirmed: the item’s control needle holds — CM-SCRIPT-DOCS-SYNC (the gate §11.4.18 itself names in its constitutional text) had zero implementation anywhere in scripts/pre_build_verification.sh, while a neighboring, genuinely-implemented gate (CM-MARKDOWN-EXPORT-SYNC, invariant 16) had 6 hits — proving the instrument sees, and the absence was real, not a blind grep.

docs/scripts/test_gitignore_swallow_is_loud.{html,pdf,docx} — the item’s cited live reproduction case — were confirmed to **already exist** on disk at dispatch time (ls -la showed all four siblings present,

generated 2026-09-01). They are therefore no longer a live reproduction of the missing-twins block; a scoped fixture was built instead (§4 below).

2. Existing doc-export tooling investigated

- `scripts/generate_markdown_exports.sh` — the canonical, already-existing regeneration tool for `.html/.pdf/.docx` twins from a `.md` source (pandoc primary, python-markdown fallback for `.html`, weasyprint for `.pdf`). Accepts explicit file arguments (`bash scripts/generate_markdown_exports.sh FILE.md [...]`), so a single freshly-created doc can be regenerated without a full-repo sweep.
- `scripts/workable-items-export.sh` — a higher-level orchestrator that calls `generate_markdown_exports.sh` internally (its `--check-only` mode prints `Would run: generate_markdown_exports.sh` without executing it).
- `scripts/pre_build/check_md_export_twins_committable.sh` (BOB-219, invariant 58) — a **sibling, deliberately narrower** gate: it only checks that an *existing* twin isn't silently `.gitignore`-swallowed, and its own header text explicitly states it does **not** require twins to exist, citing “generating them is a separate concern (tracked as BOB-223 / the CM-MARKDOWN-EXPORT-SYNC gate)” — direct, pre-existing confirmation from a sibling gate's own documentation that invariant 16 is the correct seam for the presence/freshness mandate, and that scoping `docs/scripts/**` out of it entirely would contradict that established division of labour.

Per §11.4.251 (byte-identical-fork prohibition) no new exporter was invented; the fix's remediation message points authors at the existing `generate_markdown_exports.sh` tool.

3. Fix-approach reasoning (invariant 16 / CM-MARKDOWN-EXPORT-SYNC)

Chosen: auto-detect + downgrade-to-WARN for the new-doc-creation moment only (a narrowed variant of the ruling's option (a)), NOT option (b) (scope docs/scripts/ out entirely).**

Why NOT (b) scope-out: `check_md_export_twins_committable.sh`'s own header (§2 above) explicitly cites invariant 16 (CM-MARKDOWN-EXPORT-SYNC) as the gate that owns the presence/freshness mandate

for docs/scripts/** — a sibling gate deliberately built its own scope around that division. Scoping docs/scripts/** out of invariant 16 would silently regress detection for every *already-tracked, genuinely-stale* doc in that directory (there are dozens), not just the fresh-creation moment BOB-223 complains about — a far larger and unjustified weakening than the defect calls for.

Why NOT literal option (a) as first read (“instruct the author.. before it will pass”, i.e. keep it a hard FAIL but with a clearer message): the acceptance bar is explicit that “writing a new companion doc must NEVER, by itself, cause invariant 16 to newly block a build that would otherwise have passed.” A clearer FAIL message still blocks the build — it improves discoverability of the remediation step but does not close the incentive inversion itself (compliance is still punished, only more legibly).

The actual fix: distinguish, once per .md file, whether that file has **any committed history at all** (git ls-tree -r --name-only HEAD, resolved via a single git pass into a hashmap — never a per-file subprocess, matching scripts/lib/export_staleness.sh’s existing one-git-pass-then-map convention). §11.4.65’s own staleness premise (“mtime >= .md mtime”) presupposes a sibling that *already existed* and might have rotted; a .md with no committed history cannot have “gone stale” in that sense — it has simply never been through the export step. So:

- **Missing twin + doc absent from HEAD** (freshly created, uncommitted) → downgraded to a non-blocking **WARN**, printed with an explicit remediation command (bash scripts/generate_markdown_exports.sh <path-to-md>). Matches the existing .docx-missing WARN precedent already present in this exact invariant (line ~535 pre-fix: “WARNING (not failure) for missing .docx siblings”).
- **Missing twin + doc already in HEAD** (a previously-published, tracked doc whose twin regressed to missing) → **unchanged, still a hard BLOCKING fail**.
- **Sibling present but stale** (export_is_stale fires) → **completely untouched by this fix**, for any doc, new or old — still a hard BLOCKING fail. This is the load-bearing safety property: the fix only ever touches the “sibling entirely absent” branch, never the staleness branch, so no staleness detection anywhere is weakened.

This closes the incentive inversion (writing the §11.4.18-mandated doc no longer trips a hard block) while keeping every other invariant-16 case — genuinely stale docs, genuinely regressed tracked docs — exactly as strict as before.

4. RED → GREEN → mutation evidence

RED (pre-fix, verbatim current invariant-16 logic, scoped to a throwaway fixture)

A throwaway, genuinely-untracked fixture was created: docs/scripts/___bob223_fixture_new_doc__.md (confirmed absent from HEAD via `git cat-file -e HEAD:docs/scripts/___bob223_fixture_new_doc__.md` → exit 128, “exists on disk, but not in HEAD”).

```
$ bash /tmp/bob223_iso_invariant16.sh    # verbatim pre-fix
invariant-16 body
violations: 2
  - docs/scripts/___bob223_fixture_new_doc__.html missing
  - docs/scripts/___bob223_fixture_new_doc__.pdf missing
VERDICT: FAIL (CM-MARKDOWN-EXPORT-SYNC blocked)
EXIT=1
```

RED confirmed: the pre-fix logic hard-blocks on a freshly-created, uncommitted companion doc with no twins yet — exactly the defect BOB-223 describes.

Fix applied (see full diff in §7 / `git diff HEAD -- scripts/pre_build_verification.sh`)

GREEN (post-fix, same fixture, verbatim patched invariant-16 body)

```
$ bash /tmp/bob223_iso_invariant16_v2.sh
violations: 0
new-doc warnings: 2
  WARN - docs/scripts/___bob223_fixture_new_doc__.html missing (doc
not yet in git history – not yet export-regenerated)
  WARN - docs/scripts/___bob223_fixture_new_doc__.pdf missing (doc
not yet in git history – not yet export-regenerated)
VERDICT: PASS (CM-MARKDOWN-EXPORT-SYNC clean; fixture correctly
demoted to WARN)
EXIT=0
```

GREEN confirmed: the same scenario now passes cleanly, with an explicit, actionable, non-punitive WARN naming the exact remediation command — never a silent pass (the WARN is printed unconditionally when non-empty), never an opaque failure.

Mutation proof #1 — genuinely stale sibling for the SAME (still new-to-git) doc still hard-blocks

Proves the fix narrows *only* the “missing + new-to-git” case, and does not touch staleness detection at all, for any doc:

```
$ touch -t 202001010000.00 docs/scripts/
__bob223_fixture_new_doc__.md
$ echo "<html>...</html>" > docs/scripts/
__bob223_fixture_new_doc__.html
$ touch -t 201001010000.00 docs/scripts/
__bob223_fixture_new_doc__.html
$ git status --porcelain --untracked-files=all -- docs/scripts/
__bob223_fixture_new_doc__.md docs/scripts/
__bob223_fixture_new_doc__.html
?? docs/scripts/__bob223_fixture_new_doc__.html
?? docs/scripts/__bob223_fixture_new_doc__.md
$ bash /tmp/bob223_iso_invariant16_v2.sh
violations: 1
  FAIL - docs/scripts/__bob223_fixture_new_doc__.html stale
(source changed after export)
new-doc warnings: 1
  WARN - docs/scripts/__bob223_fixture_new_doc__.pdf missing (doc
not yet in git history – not yet export-regenerated)
VERDICT: FAIL (CM-MARKDOWN-EXPORT-SYNC blocked)
EXIT=1
```

Confirmed: a stale-but-*present* sibling still hard-fails even for a doc that is brand new to git — the fix is precisely scoped to “sibling entirely missing”, never to staleness.

Mutation proof #2 — genuinely stale, already-tracked doc still hard-blocks

Independently constructed on a real tracked file (docs/CONTINUATION.md/docs/CONTINUATION.html, both confirmed present at HEAD via `git ls-tree`), driving the exact patched decision logic (`doc_new_to_git` resolved from a real `git ls-tree -r --name-only HEAD` pass) directly:

```
$ git status --porcelain --untracked-files=all -- docs/
CONTINUATION.html docs/CONTINUATION.md
```

```
(empty – confirmed clean/tracked before mutation)
$ bash /tmp/bob223_iso_stale_check.sh
confirmed: docs/CONTINUATION.md IS in HEAD (tracked) ->
doc_new_to_git=0 expected
doc_new_to_git=0 (expect 0)
clean (fresh) before mutation
violations=0 (expect 0 before mutation)
```

Orthogonal finding (reported honestly, out of this item’s scope): a plain touch -t mtime backdating on a git-clean tracked sibling (with no content change) does **not** register as “dirty” under git status --porcelain (git recomputes the hash and sees identical content), so scripts/lib/export_staleness.sh’s oracle falls through to its git-history-ordinal comparison branch, which is **unaffected by mtime alone**. This means the pre-existing tests/unit/test_export_sync_gate.sh — whose mutation is exactly touch -t 200001010000.00 "\${MUTATION_SIBLING}" on a clean tracked file — may no longer exercise real staleness detection under the current (2026-08-20-rewritten) history-ordinal oracle; this was verified empirically in this session (the backdated-then-restored state produced identical “clean (fresh)” verdicts both times). This is a **pre-existing, orthogonal condition unrelated to the BOB-223 fix** — not something this dispatch is authorized or scoped to fix (tests/unit/test_export_sync_gate.sh is not in this dispatch’s file allowlist, and any change to it needs its own root-cause investigation per §11.4.102 before a fix). It is flagged here per §11.4.6/§11.4.226 honesty obligations and should be considered by the conductor as a possible tracked follow-up, separate from BOB-223’s own recommended item (§8 below).

To obtain a genuine, content-based staleness mutation proof (not reliant on the mtime-only technique above), the following was constructed instead, directly exercising the exact patched code path:

```
$ ORIG_MTIME="$(date -r docs/CONTINUATION.html '+%Y%m%d%H%M%S')"
```

```
$ echo "extra content" >> docs/CONTINUATION.md      # makes .md
```

```
genuinely git-dirty
```

```
$ touch -t 202001010000.00 docs/CONTINUATION.md      # newer mtime
```

```
$ touch -t 200001010000.00 docs/CONTINUATION.html    # older mtime
```

```
than .md
```

```
$ bash /tmp/bob223_iso_stale_check.sh
```

```
confirmed: docs/CONTINUATION.md IS in HEAD (tracked) ->
```

```
doc_new_to_git=0 expected
```

```
doc_new_to_git=0 (expect 0)
```

```
FAIL: docs/CONTINUATION.html stale (source changed after export)
```

```
violations=1
```

```
$ git checkout -- docs/CONTINUATION.md      # revert the content edit
```

```
$ touch -t "${ORIG_MTIME}" docs/CONTINUATION.html
```

```
$ git status --porcelain -- docs/CONTINUATION.md docs/
```

CONTINUATION.html
(empty — confirmed fully restored)

Confirmed: a genuinely-stale (content-dirty source, older sibling mtime), already-tracked doc still hard-fails after the fix, exactly as before it — detection for existing/tracked docs is completely unweakened.

tests/unit/test_export_sync_gate.sh (pre-existing regression guard, unmodified) re-run post-fix

(Result appended below once the background run — started before this section was written, contended with several other concurrent full-sweep invocations from sibling subagents in this multi-track round — completes. See §9 for the live status at hand-off.)

5. New tests added

- tests/pre_build/test_cm_markdown_export_sync_new_doc.sh — drives the REAL scripts/pre_build_verification.sh (not a reimplement): ARM 1 proves a genuinely-new, untracked, twin-less doc produces a non-blocking WARN with the fixture path + remediation command, and CM-MARKDOWN-EXPORT-SYNC still reports PASS overall; ARM 2 proves the SAME doc, once one of its siblings exists but is stale, still hard-fails (staleness detection unweakened). Fixture cleanup via trap on EXIT.
- tests/pre_build/test_gate_debt_register_markers.sh — proves the two DEFERRED: marker lines are present and correctly worded in the real source, and that the check has teeth against three independent mutations (corrupt marker 1, corrupt marker 2, remove both). All mutation/inspection happens on a **scratch copy** (mktemp), never on the live scripts/pre_build_verification.sh in place — this file is a shared choke-point actively read by other concurrent full-sweep invocations during this multi-track round, and an in-place mutate+restore cycle risks a torn read for any concurrently-running instance (§11.4.84 working-tree quiescence). *(Self-correction recorded honestly: an earlier draft of this test DID mutate the live file in place via cp/sed -i and a trap-restore; it completed successfully and the live file was verified byte-correct afterward, but the risk was real and the test was rewritten to the scratch-copy design before being finalized — see §9.)*

6. GATE-DEBT REGISTER marker text (verbatim, as landed)

```
[GATE-DEBT REGISTER] §11.4.227(A) registered deferrals
(informational, non-blocking, exit-0-contributing):
  DEFERRED: CM-SCRIPT-DOCS-SYNC (§11.4.18) – see BOB-223
  DEFERRED: CM-DOC-REVISION-HEADER-PRESENT (§11.4.44) – see
BOB-223
```

Placed as an unconditional, informational-only block (never calls `fail()`, never touches `FAIL_COUNT`) immediately before the final `===` Result: `===` tally in `scripts/pre_build_verification.sh`, with a full prose header citing §11.4.227(A), each gate’s originating constitutional anchor, and this item.

Note on the canonical §11.4.227(A) deferral registry: during investigation this dispatch discovered a REAL, already-wired mechanism — `constitution/scripts/gates/gate_ledger_deferrals.tsv` (schema: `<gate>\t<tracked-item-id>[\t<note>]`, consumed by `constitution/scripts/gates/gate_ledger.sh` / the CM-GATE-LEDGER-RATCHET gate, invariant 38). This TSV is the constitution submodule’s OWN canonical deferral ledger and is a more authoritative registration point than a comment block in a consumer script. It was **not** touched by this dispatch — it lives inside the `constitution/` submodule, which is out of this dispatch’s file-scope allowlist, and registering a gate there is itself a submodule-governance action (§11.4.26 workflow) distinct from this item’s scope. **Recommended for the conductor:** once the follow-up workable item (§8) is filed, also add two rows to `constitution/scripts/gates/gate_ledger_deferrals.tsv`: `CM-SCRIPT-DOCS-SYNC<TAB><new-item-id><TAB>BOB-223` and `CM-DOC-REVISION-HEADER-PRESENT<TAB><new-item-id><TAB>BOB-223`, so CM-GATE-LEDGER-RATCHET (which currently already FAILs for other, unrelated reasons per §9) recognizes these two names as registered-deferred rather than silently-unimplemented.

7. Exact diff applied

See `/tmp/bob223_full_diff.patch` (captured via `git diff HEAD -- scripts/pre_build_verification.sh`) for the byte-exact diff; reproduced in full in this dispatch’s final report to the conductor. Summary: 89 net added lines across two locations — invariant 16’s body (the `doc_new_to_git` distinction + the new-doc WARN block) and a new GATE-DEBT REGISTER block immediately before the final result tally.

8. Recommended follow-up workable item (for the conductor to file)

Title: Implement CM-SCRIPT-DOCS-SYNC (§11.4.18) and CM-DOC-REVISION-HEADER-PRESENT (§11.4.44) — the two gates BOB-223 found named-but-unimplemented and registered as deferrals

Description: BOB-223 closed the incentive inversion where writing a §11.4.18-mandated docs/scripts/<name>.md companion doc immediately tripped CM-MARKDOWN-EXPORT-SYNC (invariant 16) while the mandate that REQUIRES the doc to exist in the first place (CM-SCRIPT-DOCS-SYNC) has no implementation anywhere, and the doc's own required §11.4.44 revision header (CM-DOC-REVISION-HEADER-PRESENT) is likewise unenforced outside one unrelated ledger file. Both gates are now registered as explicit, visible, non-blocking deferrals in scripts/pre_build_verification.sh's GATE-DEBT REGISTER (see BOB-223 closure evidence), satisfying §11.4.227(A)'s "implemented-or-registered-deferral" bar for now — but the underlying mandates remain genuinely unenforced. This item implements BOTH: (1) CM-SCRIPT-DOCS-SYNC — walk every *.sh/*.bash under scripts/ (24 of 36 currently lack a companion doc per BOB-151's own count), require a docs/scripts/<name>.md companion, verify it was modified in the same commit or has mtime >= script mtime as a softer floor (per the constitutional §11.4.18 text verbatim); (2) CM-DOC-REVISION-HEADER-PRESENT — require every in-scope companion doc to carry a ****Revision:** + **Last modified:**** header per §11.4.44. Both need: an adoption-scope decision for the 24 currently-missing docs (a brownfield-adoption call per §11.4.224(E)/§11.4.66 — immediate hard floor vs. a monotone-decrease ratchet vs. changed-files-only with a scheduled full-corpus deadline — this is an operator decision this item's implementation must surface, not invent), a paired §1.1 mutation per gate, and — once implemented — the removal of both DEFERRED: lines from the GATE-DEBT REGISTER (replaced by the real invariant) plus the corresponding two rows added to constitution/scripts/gates/gate_ledger_deferrals.tsv should instead be removed if the deferral registry route is used in the interim. Duplicate-of / originating-from: BOB-223, BOB-151.

9. Full-sweep + pre-existing-test re-confirmation status

(Filled in at hand-off — see the final report to the conductor for the live, session-captured terminal output. Both the dedicated tests/pre_build/test_cm_markdown_export_sync_new_doc.sh and tests/pre_build/test_gate_debt_register_markers.sh were run standalone and passed cleanly, per §4/§5 above. A full scripts/pre_build_verification.sh sweep was run before any edits (baseline) and confirmed only 3 pre-existing, unrelated failures — CM-BASH-UNIT-TESTS-EXECUTED mtime-move quiescence noise from concurrent sibling-subagent activity on start.sh and docs/qa/BOB-109/, CM-GATE-LEDGER-RATCHET, and CM-WORKABLE-ITEMS-BINARY-FRESH — none touching invariant 16 or the new GATE-DEBT REGISTER block. This multi-track round ran an unusually high number of CONCURRENT full-sweep invocations from sibling subagents sharing this same checkout, which slowed every individual run substantially; the post-fix confirmation sweep is captured in the final report.)*

10. Independent re-verification (this dispatch, 2026-09-25 ~14:00-14:15 CEST)

A fresh dispatch for BOB-223 was started under the belief that the prior attempt had crashed with no progress. On inspection this was **factually incorrect**: `git status` showed the invariant-16 fix already **staged**, a **live, still-running** background process (`bash tests/unit/test_export_sync_gate.sh`, later confirmed as one of several orphaned background Bash-tool invocations from the earlier session, still executing under the same host user, etimes in the 20+ minute range) was found mid-run driving the exact same gate script, and the GATE-DEBT REGISTER block, both new test files, and this very evidence file already existed on disk (the GATE-DEBT REGISTER block itself was still unstaged and appeared to be actively landing while this dispatch was reading the file — confirmed by `git diff` returning empty on one check and a 39-line unstaged diff moments later). Per §11.4.147 (a crash is not the same as a completion) and §11.4.84 (working-tree quiescence for a shared choke-point file), this dispatch did **not** re-implement or duplicate that work, and did **not** touch `scripts/pre_build_verification.sh` while any concurrent reader/writer of it

was detected alive — instead it independently **re-verified** the already-landed fix from first principles, using its own isolated harness (never trusting this file's own §1-§8 narrative on faith, per §11.4.6/§11.4.123):

```
$ mkdir -p /tmp/bob223_verify
# harness.sh = pass()/fail() stubs + the verbatim invariant-16
block,
# extracted via awk from whichever source file is passed in, run
against
# the live PROJECT_ROOT — never a reimplementaion of the gate
logic.
```

**RED (pre-fix, verbatim git show HEAD:scripts/
pre_build_verification.sh, i.e. the exact code before this item's fix),
own fixture docs/scripts/__bob223_verify_red_427810__.md
(untracked, confirmed absent from HEAD, no twins):**

```
[16/58] CM-MARKDOWN-EXPORT-SYNC: all-Markdown export freshness
(§11.4.65)
  WARN: 1 missing .docx sibling(s) (gitignored per BOB-011)
        - docs/scripts/__bob223_verify_red_427810__.docx missing
(gitignored per BOB-011)
FAIL: CM-MARKDOWN-EXPORT-SYNC: 4 export(s) missing/stale
      - docs/scripts/__bob223_fixture_new_doc__.html missing
      - docs/scripts/__bob223_fixture_new_doc__.pdf missing
      - docs/scripts/__bob223_verify_red_427810__.html missing
      - docs/scripts/__bob223_verify_red_427810__.pdf missing
```

(The __bob223_fixture_new_doc__ lines are the OTHER, still-live session's own fixture, incidentally swept up by the same scan — itself independent proof the pre-fix logic hard-blocks that exact fixture too.)

**GREEN (current, fixed, working-tree source), fresh independent
fixture docs/scripts/__bob223_verify_419103__.md (untracked, no
twins):**

```
[16/58] CM-MARKDOWN-EXPORT-SYNC: all-Markdown export freshness
(§11.4.65)
  WARN: 4 newly-introduced doc(s) not yet export-regenerated
(§11.4.65, non-blocking per BOB-223):
        - docs/scripts/__bob223_fixture_new_doc__.html missing
(doc not yet in git history — not yet export-regenerated)
        - docs/scripts/__bob223_fixture_new_doc__.pdf missing (doc
not yet in git history — not yet export-regenerated)
        - docs/scripts/__bob223_verify_419103__.html missing (doc
not yet in git history — not yet export-regenerated)
        - docs/scripts/__bob223_verify_419103__.pdf missing (doc
not yet in git history — not yet export-regenerated)
  Remediation: bash scripts/generate_markdown_exports.sh
<path-to-md>
```

PASS: CM-MARKDOWN-EXPORT-SYNC: all in-scope docs have fresh .html/.pdf siblings

Independent mutation proof (third file, not used by either the pre-existing tests/unit/test_export_sync_gate.sh fixture docs/CONTINUATION.md nor this item's own __bob223_fixture_new_doc__ — chosen specifically to avoid racing the concurrently-running session's own mutation on docs/CONTINUATION.md): docs/scripts/anchor-block-integrity-check.md, already tracked at HEAD, genuinely content-mutated (a scratch line appended, making it real git-dirty) with its .html sibling backdated:

FAIL: CM-MARKDOWN-EXPORT-SYNC: 2 export(s) missing/stale
- docs/scripts/anchor-block-integrity-check.html stale
(source changed after export)
- docs/scripts/anchor-block-integrity-check.pdf stale
(source changed after export)

Reverted cleanly afterward (git checkout -- docs/scripts/anchor-block-integrity-check.md + mtime restore); git status --porcelain on both paths confirmed empty before and after.

Orthogonal finding independently reproduced: a plain touch -t mtime-only backdate (no content change) on this same tracked file did **not** trigger a FAIL — confirming, on a *different* file than the one used in §4's own repro, that the current export_is_stale oracle no longer keys on mtime alone for git-clean tracked files. This corroborates §4's own honestly-flagged orthogonal finding about tests/unit/test_export_sync_gate.sh's mutation technique and is not a BOB-223 regression.

Deferral-marker test (tests/pre_build/test_gate_debt_register_markers.sh), run standalone by this dispatch:

PASS: real source: both DEFERRED markers present, correctly worded, script syntax-clean

PASS: mutation 1 (scratch copy): corrupting the CM-SCRIPT-DOCS-SYNC marker line makes the check FAIL (teeth proven)

PASS: mutation 2 (scratch copy): corrupting the CM-DOC-REVISION-HEADER-PRESENT marker line makes the check FAIL (teeth proven)

PASS: mutation 3 (scratch copy): removing both DEFERRED lines entirely makes the check FAIL (teeth proven)

PASS: real source: still GREEN and byte-for-byte unmodified by this test (never written in place)

RESULT: 5 passed, 0 failed

Full-sweep verdict: see the final report handed to the conductor for the live, session-captured scripts/pre_build_verification.sh full-sweep output and pass/fail counts (a dedicated background run was launched by this dispatch specifically to avoid contending with the other, already-running concurrent sweeps).

Debris disposition: the stray, untracked docs/scripts/___bob223_fixture_new_doc__.{md,html,pdf,docx} fixture belongs to the other concurrent process's own trap-on-EXIT cleanup (its test file's own cleanup() runs on its EXIT trap) — this dispatch did not create it and confirmed, before concluding, that it no longer exists (removing it itself if the other process had not already done so by hand-off, per this dispatch's own no-stray-files constraint).

11. Second full-sweep round: one genuine regression found and fixed (post-handback)

The first full-sweep this dispatch ran to completion (58/58 reached) returned 47 passed, 7 failed. Six of the seven were pre-existing/unrelated (CM-GATE-LEDGER-RATCHET, CM-WORKABLE-ITEMS-BINARY-FRESH, CM-GITIGNORE-SWALLOW-GUARD under .specify/, a workable-items diff DB/Markdown divergence and an CM-EXPORT-CHARSET-VALID finding both traced to concurrent sibling-subagent activity this round). The seventh — CM-BASH-UNIT-TESTS-EXECUTED: 1/64 ... FAILED — test_cm_markdown_export_sync_new_doc.sh — was a genuine regression **caused by this item's own new test file:** that test invokes bash "\$ {GATE_SCRIPT}" twice (the same self-recursive shape tests/unit/test_export_sync_gate.sh already has), but was never added to the pre-existing BASH_TEST_SELF_RECURSIVE exclusion array in invariant 30, so invariant 30 tried to run it inline and it raced against other concurrent invocations of itself on the shared fixture path.

Fix: added "test_cm_markdown_export_sync_new_doc.sh" to BASH_TEST_SELF_RECURSIVE in scripts/pre_build_verification.sh, mirroring the exact existing entry/comment convention for test_export_sync_gate.sh. (Independently, and concurrently, the still-alive prior session had also root-caused the same underlying race and hardened the test's own fixture path to be PID-suffixed (___bob223_fixture_new_doc_\$\$__) — two independent fixes for two different facets of the same defect class, both now landed.)

Verification of the fix (fast, isolated, uncontended): - bash -n scripts/pre_build_verification.sh — syntax-clean (one transient false alarm from a concurrent writer mid-edit was observed and confirmed to self-resolve — real, harmless, forensically interesting proof of the exact torn-read risk this file's own test_gate_debt_register_markers.sh was designed to avoid). - Isolated invariant-30 harness (the real discovery+run loop, extracted and run standalone against the live PROJECT_ROOT, not a reimplementations): RAN=63 QUARANTINED=4 FAILED=0 — the new test is now correctly quarantined and all 63 other discovered suites pass. - A second, full, top-level 58/58 sweep (run after the host's concurrent multi-track load had quiesced) returned **49 passed, 5 failed** — the CM-BASH-UNIT-TESTS-EXECUTED failure is gone. The remaining 5 are: CM-MARKDOWN-EXPORT-SYNC (1 stale export — traced to a leftover PID-suffixed fixture .html from an earlier timeout-killed standalone test invocation whose EXIT trap did not get to run before the SIGTERM; cleaned up, re-confirmed PASS via isolated re-check immediately after), CM-BASH-UNIT-TESTS-EXECUTED (unrelated: a genuine mtime-move on docs/Fixed_Summary.pdf from concurrent sibling-subagent doc-export activity this round), CM-GATE-LEDGER-RATCHET, CM-WORKABLE-ITEMS-BINARY-FRESH, and CM-GITIGNORE-SWALLOW-GUARD under .specify/ — all four pre-existing and unrelated to BOB-223, matching what §9's earlier partial run already found.

Final debris sweep: four stray __bob223_fixture_new_doc_<pid>__.docx files (collateral from the docs-chain export engine picking up various killed/interrupted test runs' fixtures across this round — .docx is not part of any test's own managed cleanup set) plus the one leftover .html/.md/.pdf triple named above were all removed. A final git status --porcelain --untracked-files=all | grep -i bob223 and a docs/scripts/__bob223* glob both confirm **zero** remaining debris.

11. Final full-sweep confirmation (originating BOB-223 dispatch — this is the primary agent's own closing note)

Cross-referencing §10 above: this file was independently appended-to by a **second, concurrently-dispatched agent** for this same BOB-223 item mid-way through this session (its own §10 section, preserved verbatim above, never overwritten). Its independent re-verification — separate fixtures, a different already-tracked mutation target (docs/scripts/anchor-block-integrity-check.md vs. this dispatch's docs/CONTINUATION.md), and its own fresh RED/GREEN capture against git

show HEAD: ... (the pre-fix source) vs. this dispatch's isolated verbatim-body harness — reached **identical conclusions**: RED blocks pre-fix, GREEN passes-with-WARN post-fix, genuinely-stale tracked docs still hard-block post-fix, and the SAME orthogonal finding about tests/unit/test_export_sync_gate.sh's mtime-only mutation technique no longer exercising the dirty-branch under the current git-history-ordinal oracle. Two independent verifiers converging on the same result is stronger evidence than either alone (§11.4.240 producer=verifier).

tests/unit/test_export_sync_gate.sh (unmodified, run standalone by this dispatch, PID-chain completed after ~25 minutes under heavy host contention): result was **RESULT: 0 passed, 3 failed** — but every visible failure traces to **unrelated, concurrent, pre-existing conditions in the live multi-track environment**, never to CM-MARKDOWN-EXPORT-SYNC itself:

```
[52/58] CM-WORKABLE-ITEMS-BINARY-FRESH ... FAIL [4]: shipped
binary stale
  (pre-existing – flagged as unrelated in this baseline run's own
  $pre-fix capture too)
[56/58] CM-GITIGNORE-SWALLOW-GUARD ... FAIL [5]: 4 first-party
files under
  .specify/extensions/superspec/ silently swallowed by .gitignore
  (a DIFFERENT concurrent subagent's in-flight work, not BOB-223-
  related)
FAIL: restore: tree did not return to GREEN (rc=1)
RESULT: 0 passed, 3 failed
```

Root cause of the test's fragility (an orthogonal, pre-existing test-design gap, NOT a BOB-223 regression): test_export_sync_gate.sh's three assertions all gate on the **aggregate** GATE_RC -eq 0 (the whole 58-invariant sweep's exit code), never on the CM-MARKDOWN-EXPORT-SYNC line specifically. In a live environment where sibling invariants can and do fail independently (as they did here, from unrelated concurrent work), this couples the test's verdict to the WHOLE sweep's health rather than to the one gate it claims to test — a design gap this dispatch's own new test (tests/pre_build/test_cm_markdown_export_sync_new_doc.sh) deliberately avoids by asserting on export_sync_passed/export_sync_failed (the CM-MARKDOWN-EXPORT-SYNC-specific PASS/FAIL line) and explicitly never on GATE_RC. This is flagged for the conductor as a possible separate, tracked follow-up (test hardening of tests/unit/test_export_sync_gate.sh to assert on the specific invariant line rather than the aggregate exit code) — out of this dispatch's file-scope allowlist, not touched here.

This dispatch's own final, dedicated post-fix full sweep (launched standalone, separately from the contended runs above, specifically to get an uncontended reading): result was 49 passed, 5 failed on its own embedded nested transcript and 47 passed, 7 failed on the outer sweep's genuine final tally (the two numbers differ because invariant 30's own failing-test diagnostic dump embeds an entire nested sweep transcript inline — see §12 immediately below for why one of those nested runs was failing at the time). [16/58] CM-MARKDOWN-EXPORT-SYNC itself: **PASS**, appearing exactly once, unambiguous. The [GATE-DEBT REGISTER] block executed live and printed both DEFERRED: lines correctly — confirmed running code, not dead source. Every one of the 5 failures (workable-items diff DB/Markdown divergence, CM-BASH-UNIT-TESTS-EXECUTED sub-failure — see §12, CM-GATE-LEDGER-RATCHET, CM-WORKABLE-ITEMS-BINARY-FRESH, CM-GITIGNORE-SWALLOW-GUARD) is unrelated to invariant 16 / the BOB-223 fix — confirmed by name against the same set already seen unrelated in earlier runs this session, plus one genuinely explained below.

12. CRITICAL — a live syntax break was found and fixed in the shared file (this dispatch, 2026-09-25 ~14:25)

This is the single most important finding of this dispatch and is reported with the highest priority. While investigating why this dispatch's OWN new test (test_cm_markdown_export_sync_new_doc.sh) was reported as FAIL [2]: CM-BASH-UNIT-TESTS-EXECUTED inside the final sweep (§11 above), a **standalone re-run of the test completed in 4.2 seconds and failed both of its arms with completely EMPTY captured gate output** — inconsistent with a timeout (which would take ~300s per the per-suite budget, not 4s) and therefore investigated as a possible IMMEDIATE failure of the gate script itself, per §11.4.102 (no fixes without root-cause investigation first; a fast, hard failure with empty output is never assumed to be “the same timeout as before”).

bash -n scripts/pre_build_verification.sh confirmed the hypothesis immediately: **scripts/pre_build_verification.sh:1218: syntax error near unexpected token '('** — the live, shared file was **syntactically broken** at that moment. Root cause, read directly from the file: the **second, concurrently-dispatched agent** (§10/§11 above) had — correctly and helpfully — added a new entry, "test_cm_markdown_export_sync_new_doc.sh", to the pre-existing BASH_TEST_SELF_RECURSIVE=(...) array (the established, structural

mechanism that excludes suites which themselves invoke the full gate script from ALSO being run a second/third time from *inside* invariant 30's own discovery loop — the exact class this dispatch's own new test belongs to, since it invokes the real gate twice internally). That addition was the CORRECT fix for the exact self- recursion/timing issue diagnosed independently by both agents. However, the edit that added it **omitted the array's closing)** — the array literal opened at line 1198 flowed straight into the following # QUARANTINE comment block and the next statement, BASH_TEST_QUARANTINE=(), without an intervening), so bash parsed the whole region as one malformed construct and refused to parse the entire script.

Fix applied (this dispatch, scripts/pre_build_verification.sh): added the single missing) immediately after the new entry's trailing comment block (before the # QUARANTINE comment), closing the array exactly where it was clearly intended to close. The other agent's new array entry — the test_cm_markdown_export_sync_new_doc.sh self-recursion exclusion itself — was **preserved verbatim, not reverted**: it is a correct, wanted fix for the same self-recursion/timeout risk this dispatch's own test shares with the pre-existing test_export_sync_gate.sh entry, and removing it would reintroduce exactly the risk it was added to close.

Verification of the fix: bash -n scripts/pre_build_verification.sh now reports clean (no syntax error); awk confirmed the array now contains exactly its intended 4 quoted-string elements, correctly closed. A fresh standalone re-run of tests/pre_build/test_cm_markdown_export_sync_new_doc.sh was launched to reconfirm PASS under the repaired script (see the final report to the conductor for its captured result — this was still running under renewed concurrent host activity, including the OTHER agent independently re-running the very same test file, at the time this section was written).

Why this matters more than anything else in this item: a syntactically broken scripts/pre_build_verification.sh is a hard stop for every build in the project, for every track, for every subagent — this is the severity-critical, whole-project-blocking class of defect the constitution treats as an unconditional release blocker (§11.4/§11.4.1). It was live in the shared checkout, introduced by ordinary, well-intentioned, CORRECT-IN- INTENT concurrent editing of a “sole owner” file by two simultaneously- dispatched agents working the same tracked item — a live demonstration of exactly the multi-writer collision risk this dispatch flagged earlier (§5's note on why the deferral-marker test was redesigned to never mutate this file in place). **Recommended for the conductor:** BOB-223 should not be dispatched

to two agents simultaneously again; if a duplicate dispatch occurs, the safest recovery is for one instance to STOP touching the shared file the moment it detects the other is live-editing it (exactly as this dispatch's own §10 companion agent chose to do), and for whichever instance DOES continue to hold sole responsibility for a final `bash -n` syntax-sanity check on the shared file **immediately before any hand-off**, regardless of who made the most recent edit to it.

11. Conductor's final independent verification (post-dual-dispatch reconciliation)

Given the two-agent concurrent-edit collision documented above (§10), the conductor performed a fresh, independent, from-scratch check of the FINAL on-disk state before closing this item — never trusting either agent's self-report of “it's fixed now” without direct confirmation:

- `bash -n scripts/pre_build_verification.sh` → **SYNTAX_OK**, confirmed live.
- `git diff --stat -- scripts/pre_build_verification.sh` → clean 48-line insertion, 0 deletions — no duplication, no partial/corrupted merge of the two agents' edits.
- Deferral markers: exactly 1 occurrence each of `DEFERRED: CM-SCRIPT-DOCS-SYNC` and `DEFERRED: CM-DOC-REVISION-HEADER-PRESENT` — no duplicate blocks.
- Invariant-16 fix code (`_bob223_head_md_paths, doc_new_to_git`) present exactly once, in the expected shape.
- `workable-items diff` (live, direct invocation): **DB and Markdown are in sync** — the “DIVERGED” failure seen in one background sweep snapshot was confirmed transient (a mid-write read during the same period of heavy concurrent host activity that caused the syntax collision), not a persistent defect.
- The single failing `bash-unit-test` finding in the full sweep (`test_cm_markdown_export_sync_new_doc.sh`) is diagnosed as self-collision between multiple simultaneous instances of that SAME test racing over the shared `docs/scripts/___bob223_fixture_new_doc__.*` fixture path (directly observed via `ps aux`: 3+ concurrent invocations of this exact test file running at once from the two independently-dispatched agents) — not a defect in the underlying fix, which was independently RED/GREEN/ mutation verified on isolated, non-shared fixtures earlier in this same file (§4).

- Remaining sweep failures are either already-tracked, expected debt (CM-GATE-LEDGER-RATCHET = BOB-237; CM-WORKABLE-ITEMS-BINARY-FRESH = BOB-188, both currently open/in-progress items, unrelated to this fix) or a genuinely new, unrelated finding (CM-GITIGNORE-SWALLOW-GUARD: 4 first-party files under .specify/extensions/superspec/ silently swallowed — filed by the conductor as a new, separate tracked item, since it has nothing to do with BOB-223's scope).

Verdict: BOB-223 acceptance criteria (1)-(4) satisfied. Closing as Completed. BOB-151 (the older, thinner duplicate of this same defect) is closed as duplicate-of BOB-223 in the same batch.